

GatKeeper

Plateforme d'Intelligence de Sécurité AI-Native

Document d'Analyse & Architecture du Projet

Version 1.0 | Juin 2026

Auteur : Karla Albert <karlalbert152@gmail.com>

Statut : Phase 1 — Architecture MVP

CONFIDENTIEL

Table des Matières

1. Résumé Exécutif	3
2. Problématique & Analyse du Marché	4
3. Solution & Positionnement Stratégique	5
4. Architecture — Design 4 Couches	6
5. Stack Technique & Justifications	7
6. Phase 1 MVP — Structure des Dossiers	8
7. Système d'Analyse Multi-Agents	9
8. Plongée dans le Subconscious Engine	10
9. Analyse Concurrentielle	11
10. Spécification du Format .gat	12
11. Roadmap Phase 1 (3 Mois)	13
12. Monétisation & Stratégie Go-to-Market	14

1. Résumé Exécutif

GatKeeper est une plateforme d'intelligence de code de nouvelle génération conçue pour répondre à l'une des problématiques les plus critiques de notre ère numérique : la sécurité des logiciels à grande échelle. Contrairement aux outils existants qui se contentent de détecter des problèmes déjà présents, GatKeeper adopte une approche radicalement différente : anticiper, simuler et prédire les failles avant qu'elles n'existent.

La plateforme combine une architecture multi-agents basée sur des modèles d'IA spécialisés, un moteur de simulation autonome (le Subconscious Engine), et une mémoire évolutive du codebase pour offrir une couverture de sécurité sans précédent. GatKeeper s'adresse aussi bien aux développeurs individuels qu'aux grandes organisations (Fortune 500, institutions financières, gouvernements) ayant des exigences de conformité strictes (GDPR, SOC2, ISO27001, HIPAA).

Chiffres Clés

Indicateur	Valeur	Contexte
Scénarios simulés / nuit	48 000+	Par le Subconscious Engine
Agents IA spécialisés	6 agents	Chacun fine-tuné sur son domaine
Projection temporelle	30 jours	Simulation prospective du code
Langages supportés	15+	C, C++, Rust, Python, JS, Go...
Taille de marché adressable	25 Mds USD	Sécurité logicielle mondiale 2025
Économie client estimée	80%	des coûts vs audit externe classique

2. Problématique & Analyse du Marché

2.1 La Crise Silencieuse de la Sécurité Logicielle

Nous vivons une époque paradoxale : jamais les outils de développement n'ont été aussi puissants, et pourtant les failles de sécurité logicielle atteignent des niveaux records. En 2024, le coût moyen d'une violation de données dépassait 4,5 millions de dollars par incident. Les grandes organisations font face à plusieurs problèmes structurels simultanés :

- Codebases massifs (millions de lignes) impossibles à auditer manuellement
- Dizaines d'équipes développant en parallèle créant des interactions imprévues
- Dépendances externes (npm, crates.io, PyPI) introduisant des vulnérabilités tierces
- Audits de sécurité externes coûteux (100K à 500K par audit) et peu fréquents
- Outils fragmentés qui ne se parlentnt : SAST, DAST, SCA, scanners de secrets
- Conformité réglementaire (GDPR, SOC2, HIPAA) de plus en plus exigeante

2.2 Les Limites des Outils Actuels

Les solutions existantes souffrent d'un défaut fondamental : elles sont réactives. Elles trouvent ce qui est déjà cassé, jamais ce qui va l'être :

- SonarQube et outils SAST : analyse statique sans compréhension sémantique
- Snyk et outils SCA : focalisés sur les dépendances, ignorent la logique interne
- Outils de pentest manuel : coûteux, ponctuels, non intégrés au cycle de dev
- GitHub Copilot et assistants IA : suggèrent du code mais n'auditent pas
- Aucun outil ne combine mémoire évolutive, simulation prospective et analyse multi-couches

3. Solution & Positionnement Stratégique

3.1 La Vision Fondatrice

GatKeeper est la première plateforme d'intelligence de code vivante — un système qui apprend l'ADN de votre codebase, simule des milliers de futurs possibles en arrière-plan, et alerte proactivement avant que les problèmes n'arrivent. Comme un médecin qui prédit la maladie avant les symptômes plutôt que de la traiter une fois déclarée.

GatKeeper unifie en une seule plateforme ce qui nécessite aujourd'hui 5 à 8 outils distincts : analyse statique, analyse de dépendances, scan de secrets, modélisation des menaces, compliance automatique, simulation de charge et simulation d'attaque.

3.2 Positionnement Marché

Dimension	SAST classique	Assistants IA	GatKeeper
Approche	Réactive	Assistive	Prédictive & proactive
Compréhension	Syntaxique	Contextuelle partielle	Sémantique + intentionnelle
Mémoire projet	'	'	'
Simulation futur	'	'	'
100% on-premise	Partiel	'	'
Multi-agents	'	'	'
Compliance auto	Partiel	'	'

4. Architecture — Design 4 Couches

La plateforme s'articule autour de quatre couches principales, chacune avec des responsabilités clairement définies et des composants intégrés de la stack technique.

[illegible]

Responsabilités des Composants

- **INTERFACE** : Point d'entrée pour les développeurs. CLI pour les workflows terminal, plugins IDE pour le feedback temps réel, Dashboard pour la visibilité d'équipe.
- **INTÉGRATION** : Automatise les vérifications de sécurité dans le cycle de développement. Chaque commit déclenche une analyse.
- **INTELLIGENCE** : Cerveau principal. Les agents analysent, le Subconscient simule, le DNA se souvient, Tree-sitter parse.
- **DONNÉES** : Stockage persistant. SQLite pour local/portable, PostgreSQL pour l'échelle entreprise, bases CVE pour le renseignement sur les menaces.

5. Stack Technique & Justifications

Composant	Technologie	Version	Justification
CLI principal	Rust	1.75+	Performances natives, sécurité mémoire garantie
Parsing du code	Tree-sitter	0.22+	Multi-langage (15+), robuste, incrémental, embarquable
Orchestration agents	Tokio (async Rust)	1.35+	Millions de tâches concurrentes sans overhead
LLMs d'analyse	Ollama (local)	Latest	Zéro coût API pour le subconscient nocturne
LLMs cloud (Pro/Ent.)	Anthropic Claude	claude-3	Qualité d'analyse supérieure pour les tiers payants
Dashboard web	React + TypeScript	18+/5+	Interface riche, temps réel via WebSockets
Backend API	Axum (Rust)	0.7+	Cohérence avec le CLI, performances HTTP excellentes
Stockage local	SQLite (rusqlite)	3.45+	Léger, portable, versionnable, pas de serveur requis
Stockage entreprise	PostgreSQL	16+	Scalabilité, transactions, full-text search
Analyse CVE	OSV + NIST NVD API	Latest	Bases officielles, open source, MAJ temps réel
Sandbox attaques	gVisor + Docker	Latest	Isolation kernel-level pour exécution sécurisée PoC
Communication temps réel	WebSockets	Latest	Dashboard live, alertes push immédiates
Format rapport	TOML	1.0	Lisible humainement, parsable machine, versionnable Git

6. Phase 1 MVP — Structure des Dossiers

La structure de la Phase 1 MVP suit un modèle Cargo workspace avec des crates clairement séparées pour chaque composant majeur.

```
gatkeeper/
% % % Cargo.toml
% % % rust-toolchain.toml
% % % LICENSE
% % % README.md / README_FR.md
% % % .gitignore
%
% % % crates/
% % % % % gatkeeper-cli/
% % % % % % % src/
% % % % % % % main.rs
% % % % % % % commands/
% % % % % % % output.rs
% % % % % gatkeeper-core/
% % % % % % % src/
% % % % % % % config.rs
% % % % % % % error.rs
% % % % % % % project.rs
% % % % % % % report/
% % % % % gatkeeper-parser/
% % % % % % % src/
% % % % % % % languages/
% % % % % % % ast.rs
% % % % % % % walker.rs
% % % % % gatkeeper-agents/
% % % % % % % src/
% % % % % % % orchestrator.rs
% % % % % % % traits.rs
% % % % % % % security/
% % % % % % % secrets/
% % % % % % % supply_chain/
% % % % % gatkeeper-subconscious/
% % % % % % % src/
% % % % % % % engine.rs
% % % % % % % scheduler.rs
% % % % % % % threat/
% % % % % % % chaos/
% % % % % % % evolution/
% % % % % gatkeeper-dna/
% % % % % % % src/
% % % % % % % fingerprint.rs
% % % % % % % patterns.rs
% % % % % % % invariants.rs
% % % % % % % baseline.rs
% % % % % % % store.rs
%
% % % gatkeeper-lenses/
% % % migrations/
% % % templates/
% % % tests/
% % % scripts/
% % % docs/
% % % .github/workflows/

# Racine workspace

# Binaire CLI principal
# Entrée (clap)
# scan, report, init, status, whatif
# Formatage terminal
# Logique métier partagée
# Configuration projet
# Types d'erreur
# Détection projet
# .gat, risk_score, findings
# Tree-sitter multi-langage
# Rust, Python, JS, C, Go
# Wrapper AST unifié
# Patterns de parcours AST
# 6 agents IA spécialisés
# Orchestration multi-agents
# Définition du trait Agent
# SQL/XSS, crypto, OWASP
# Détecteur de secrets hardcodés
# Vérification CVE dépendances
# Subconscious Engine
# Boucle principale (Tokio spawn)
# Planification cron-like
# Rouge : simulation attaquants
# Jaune : injection pannes
# Verte : génération code
# DNA Engine (mémoire long terme)
# Empreinte cryptographique
# Détection patterns archi
# Invariants de code
# Baseline comportementale Git
# Persistance SQLite
# Compréhension de l'intention
# Schéma SQLite
# Template .gat, exemple config
# Tests d'intégration + fixtures
# Génération PDF, helpers CI
# Documentation + PDFs
# Pipeline CI/CD
```


7. Système d'Analyse Multi-Agents

Six agents IA indépendants, chacun fine-tuné sur son domaine spécifique, travaillent en parallèle sur chaque analyse. Cette architecture multi-agents permet une profondeur impossible avec un seul modèle généraliste.

Agent	Domaine	Exemples de détections
SecurityAgent	Vulnérabilités & attaques	Injection SQL/XSS/LDAP, timing attacks, crypto faible, OWASP
LogicAgent	Bugs logiques & cohérence	Race conditions, deadlocks, edge cases, incohérences inter-m
PerformanceAgent	Efficacité & ressources	Memory leaks, N+1 queries, allocations inutiles, hot paths
ComplianceAgent	Conformité réglementaire	GDPR Art.32, SOC2, ISO27001 A.12, HIPAA HITECH
SecretAgent	Secrets & credentials	API keys hardcodées, tokens JWT exposés, mots de passe en cl
SupplyChainAgent	Sécurité chaîne d'appro.	Typosquatting, dépendances orphelines, CVE tierces, licences

L'AgentOrchestrator gère l'exécution parallèle via le runtime async Tokio, collectant les findings de tous les agents et les agréant dans un rapport unifié avec scores de risque calculés.

8. Plongée dans le Subconscious Engine

8.1 Concept & Philosophie

Le Subconscious Engine est la fonctionnalité la plus innovante de GatKeeper et celle qui le distingue radicalement de tout ce qui existe sur le marché. Inspiré par le fonctionnement du cerveau humain qui continue de traiter des informations pendant le sommeil — consolidant les apprentissages, résolvant des problèmes complexes, établissant des connexions non évidentes — le Subconscious Engine opère en arrière-plan 24 heures sur 24.

8.2 Les Trois Couches du Subscient

- ROUGE — Threat Simulation : Simule des attaquants réels opérant contre votre système. Modélise différents profils d'adversaires et calcule les chemins d'attaque les plus probables.
- JAUNE — Chaos Engineering : Simule des conditions adverses pour identifier les points de rupture cachés dans l'architecture. Inspiré du Chaos Engineering de Netflix et des pratiques SRE de Google.
- VERTE — Code Evolution : Génère et évalue des alternatives améliorées au code existant. Agit comme un pair-programmer IA qui travaille pendant que l'équipe dort.

8.3 Cycles de Simulation

Phase	Fréquence	Durée	Contenu
Scan Léger	Toutes les heures	~2 min	Nouveaux commits, alertes critiques, vérif. secrets
Nightly	Chaque nuit	~45 min	Projection 7 jours, MAJ DNA, corrélation KB
Deep Sim.	Chaque semaine	~6 heures	Projection 30 jours, rééval. architecture complète
Grand Audit	Chaque trimestre	~24 heures	Sim. exhaustive, rapport conformité, recs stratégiques

9. Analyse Concurrentielle

Fonctionnalité	SonarQube	Snyk	Veracode	Semgrep	GatKeeper
Analyse statique (SAST)	'	Partiel	'	'	'
Analyse dépendances (SCA)	Partiel	'	'	Partiel	'
Scan de secrets	Partiel	'	Partiel	'	'
IA multi-agents	'	'	'	'	'
Mémoire évolutive (DNA)	'	'	'	'	'
Subconscious Engine 24/7	'	'	'	'	'
Simulation 30 jours	'	'	'	'	'
Sandbox d'attaque (PoC)	'	'	Partiel	'	'
Mode What-If	'	'	'	'	'
Compliance automatique	Partiel	Partiel	'	'	'
Knowledge Base collective	'	'	'	'	'
Cross-repository	'	Partiel	'	'	'
Prép. post-quantique	'	'	'	'	'
100% on-premise	'	'	'	'	'
Rapport versionnable	Partiel	'	'	'	'

10. Spécification du Format .gat

Le fichier .gat est bien plus qu'un simple rapport de sortie. C'est un artefact structuré, versionnable et exploitable en machine qui documente l'intégralité de l'état de santé d'un projet à un instant T. Conçu pour être attaché aux commits Git, intégré dans les pipelines CI/CD et consommé par des outils tiers via API.

```
[session]
organisation = "TechCorp SA"
project = "payment-service"
version = "2.4.1"
timestamp = "2026-06-03T08:00:00Z"
agents_utilises = ["SecurityAgent", "LogicAgent", "ComplianceAgent"]
lignes_analysees = 847_293
scenarios_sub = 48_293
duree_analyse = "6h32m (arrière-plan)"

[risk_score]
global = 73/100
securite = 61/100
performance = 88/100
compliance = 79/100
tendance = "down (-7pts sur 14 jours)"

[[findings.critical]]
id = "GAT-CRIT-001"
agent = "SecurityAgent"
fichier = "src/auth/token.rs"
lignes = [234, 267]
type = "timing_attack"
cvss_score = 8.7
description = "Comparaison token non-constant-time, exploitable"
correction = "Utiliser subtle::ConstantTimeEq"
effort_correction = "15 minutes"

[[subconscious.dreams]]
id = "DREAM-001"
couche = "threat"
decouverte_a = "03:47 AM"
horizon = "23 jours"
probabilite = 94
impact = "accès admin non autorisé"
condition = "trafic > 10k req/s"

[subconscious.intuitions]
items = [
  "auth/ et payments/ divergent silencieusement.",
  "crypto-lib abandonnée. Probabilité CVE 60j: 71%."
]
```

11. Roadmap Phase 1 (3 Mois)

Mois 1 — Fondations

- Mise en place du workspace Cargo avec toute la structure de crates
- Implémentation du CLI avec clap (init, scan, report, status)
- Intégration du parser Tree-sitter pour 5 langages (Rust, Python, JS, C, Go)
- Implémentation basique du SecurityAgent (injection, crypto, OWASP)
- Implémentation du SecretAgent avec détection de credentials hardcodés
- Génération du rapport .gat au format TOML
- Schéma SQLite pour le stockage du DNA Engine

Mois 2 — Intelligence

- Complétion des 6 agents avec analyse basée sur les patterns
- Implémentation de l'AgentOrchestrator avec exécution parallèle Tokio
- Subconscious Engine basique avec cycle horaire et projection 24h
- DNA Engine v1 : fingerprinting, baseline, détection de patterns
- SupplyChainAgent avec intégration API OSV/NVD pour vérification CVE
- Intégration CI/CD GitHub Actions

Mois 3 — Polish & Lancement

- Plugin Neovim basique
- Documentation complète et guide de démarrage
- Suite de tests d'intégration avec fixtures de projets vulnérables
- Optimisation des performances et durcissement de la gestion d'erreurs
- README, guide de contribution et mise en place de la communauté
- Lancement du dépôt GitHub public

12. Monétisation & Stratégie Go-to-Market

Plans Tarifaires

Plan	Cible	Prix	Fonctionnalités clés
Community	Dév. individuels, OSS	Gratuit	1 projet, CLI, SecurityAgent basique, .gat, sub. 24h
Pro	Freelances, startups	49€/mois	Projets illimités, 6 agents, dashboard, CI/CD, sub. 7j
Business	Équipes 5-50	299€/mois/équipe	Compliance, sandbox PoC, Horizon, KB, sub. 30j
Enterprise	Grandes orgs 50+	50K-500K\$/an	On-premise, fine-tuning, Telepathy, Quantum, SLA

Stratégie Go-to-Market

- Phase 1 : Community open source !' adoption organique par les développeurs
- Phase 2 : Product-led growth !' upgrade naturel vers Pro via feature gates
- Phase 3 : Sales-led growth !' équipe commerciale ciblant les CISO
- Partenariats stratégiques : intégrateurs (Accenture, Deloitte), cloud providers
- Conférences : DEF CON, Black Hat, RSA Conference, KubeCon

GatKeeper

Le premier outil de sécurité qui pense en permanence.

Document généré par l'Analyse de Projet GatKeeper
Version 1.0 — Juin 2026

"Pendant que tu dormais, j'ai trouvé 3 choses que tu dois savoir."